

gered by a falling/rising edge or a low-level signal at INT0, INT1 and INT2 pins. When the external interrupt is enabled and configured as low-level-triggered interrupt (only INT0/INT1), the interrupt triggers as long as the pin is held low. Note that INT2 is only an edge-triggered interrupt. These settings are done in the MCU control register (MCUCR) and MCU control and status register (MCUCSR).

In the `external_int.c` source code, once again `timer0/counter0` module is employed in the CTC mode and the compare match interrupt routine utilised. External interrupt pins INT0 and INT1 are configured to generate an interrupt request for every falling edge. Inside the external interrupt service routine, `timer0` is paused, the rate of the LED chaser updated and `timer0` resumed to its normal operation.

Example 8: EEPROM

The ATmega32 contains 1024 bytes of data EEPROM. It is organised as a separate data space in which single bytes can be read and written. The EEPROM has an endurance of at least 100,000 write/erase cycles.

With the `EEPROM.c` source code provided here you can test the EEPROM for the following operations:

1. Write all the 1024 locations in the EEPROM with a dummy value of 0x32
2. Read all the 1024 locations in the EEPROM one by one
3. Erase the entire EEPROM and store at each location a default value of 0xFF

You can perform the above write, erase and read operations by pressing switches S1, S2 and S3, respectively. The options are presented in the form of a menu on the LCD module interfaced. In order to initiate the EEPROM read and write cycles, built-in functions `eeprom_read_block()` and `eeprom_write_block()` are used. These functions are defined in the `eeprom.h` header file. Details of these functions could be found in the avr-libc reference manual included in this month's EFY-CD.

Example 9: 16-bit timer/counter1 and analogue comparator

The `freq_meter.c` sample code introduces you to the use of the 16-bit timer/counter1 module and the analogue comparator module in ATmega32.

16-bit timer/counter1. The 16-bit timer/counter1 module is similar to the 8-bit timer0/counter0 that we have discussed earlier, except that it has 16-bit resolution and two independent output compare units and an input capture unit with inbuilt input capture and noise canceller.

The 16-bit timer/counter1 is made to operate in the normal mode. A time stamp is produced by the input capture unit whenever a falling edge is detected on the output of the comparator. The time interval between two consecutive time stamps corresponds to the time period of the input square wave and therefore used to calculate the frequency of the applied signal.

Before calculating the frequency, an average of the eight samples of the time period is taken so as to minimise error in the frequency calculated. The samples are taken after ten trials so that the input signal stabilises by then.

Analogue comparator. The analogue comparator compares the input values on the positive (AIN0) and negative (AIN1) pins of the ATmega32. When the voltage on AIN0 is higher than the voltage on AIN1, the analogue comparator output (ACO) is set to trigger the timer/counter1 input-capture function. In addition, the comparator can trigger a separate interrupt, exclusive to the analogue comparator. The user can select interrupt triggering on rise, fall or toggling of the comparator output.

The analogue comparator multiplexer enable (ACME) bit in the special function IO register (SFIO) can be made high (logic 1) or low (logic 0). When this bit is written as logic 0, AIN1 input is applied to the negative input of the analogue comparator.

The `freq_meter.c` sample code measures the frequency of a square

wave applied to pin 33 (PA7) of the MCU and shows the measured value on the LCD. The ranges in Hz, kHz and MHz are to be selected by the user before the measurement using switches S1, S2 and S3, respectively. The range menu options are displayed on the LCD.

The square wave applied to PA7 is further applied to the inverting terminal of the internal analogue comparator. For this, ACME bit in the SFIO register is set, ADEN bit in the ADCSRA register cleared and MUX2.0 bits in the ADMUX set to 111.

The non-inverting terminal is provided with a bandgap reference voltage by setting ACBG bit in the ACSR register. So for every rising edge of the input square wave, a falling edge is produced at the comparator output, which is used to trigger the input capture unit of timer/counter1 module. This is ensured by clearing ICES1 bit in TCCR1B register.

Example 10: PWM operations

In this section, we will be using the 16-bit timer/counter1 module to explain the PWM operation that can be initiated using the microcontroller.

The `pwm.c` sample source code is provided to test PWM operation using the microcontroller. It does the task of controlling a servo motor. The source code provides options to rotate the servo motor through angles of 45° clockwise and 45° anti-clockwise in the form of a menu. Using switches S1, S2 and S3, you can give command to rotate the motor to angles of -45°, 0° and +45°, respectively.

Fundamentally, servo motors are DC motors with a positional feedback control that allows the rotor to be positioned accurately. The specifications state that the shaft can be positioned through a minimum of 90° (±45°). There are three wire leads to a servo motor. Of these, two are for +5V and GND connections. The third lead feeds a position control signal to the motor. The position control signal is a single